

Secure Information Exchange Program

. . .

a simulation in

DIFFIE-HELLMAN KEY EXCHANGE

with

AES-128 BLOCK ENCRYPTION

submitted by

VIOLDAN E. BAYOCOT

§ I PROJECT OVERVIEW

...

This document accompanies a Windows Forms implementation of the Secure Information Exchange Program, a simulation in which two parties—User A and User B—agree on a shared secret over an untrusted channel and use that secret to encrypt and decrypt a plaintext message. The protocol combines a classical Diffie–Hellman key exchange with symmetric AES-128 block encryption.

The implementation follows the specification closely: the public parameters are the prime $p = 199$ and the generator $g = 127$; private keys are interpreted as the decimal equivalents of ASCII characters; the computed shared key is transformed into a 128-bit AES key by a deterministic rule based on its digit count; and the plaintext is segmented into sixteen-character blocks padded with the character '@' where necessary. Each block is encrypted independently in ECB mode and concatenated to form the ciphertext transmitted to the receiver.

§ II REPOSITORY

...

The full source code, organized into the seven commits described in the build journal, is available at the public Git repository below. Each commit leaves the project in a working state and corresponds to one self-contained stage of the implementation.

GitHub Repository
<https://github.com/decr0zeath/bayocot-secure-software>

§ III

VERIFICATION AGAINST THE SPECIFICATION

...

The following test vector, drawn directly from the original specification, exercises every stage of the program. When executed, the implementation must reproduce each of the values below exactly. The screenshots that follow this section provide visual confirmation.

The values listed below are the inputs to the simulation.

<i>PARAMETER</i>	<i>VALUE</i>
Private Key of User A	57
Private Key of User B	167
Prime p	199
Generator g	127

Plaintext message: "The Mandalorian Must Always Recite, This is The Way!"

Running the simulation must reproduce exactly the values below.

<i>COMPUTED QUANTITY</i>	<i>EXPECTED VALUE</i>
Public Value of User A $(g^a \bmod p)$	17
Public Value of User B $(g^b \bmod p)$	75
Shared Key from User A $(B^a \bmod p)$	109
Shared Key from User B $(A^b \bmod p)$	109
AES-128 Key (ASCII)	109F109F109F109F

The plaintext is segmented into four 16-character sub-messages:

- [1] "The Mandalorian "
- [2] "Must Always Reci"
- [3] "te, This is The "
- [4] "Way!@@@@@@@@@@@@@" ← padded with '@'

Block four is shorter than sixteen characters in the original message and is therefore right-padded with the character '@' (binary 01000000) until it reaches the full 128-bit block length, as required by the specification.

§ IV WALKTHROUGH

...

The four screenshots below trace a single complete run of the simulation, from an empty form through key derivation, encryption, and finally decryption on the receiver's side. Each figure should match the corresponding row of the test vector in Section III.

STEP 1 • INITIAL STATE

The application has just been launched; no fields have been populated.

The screenshot shows a web application window titled "Bayocot — Secure Information Exchange". The interface is divided into two main columns: "User A (Sender)" and "User B (Receiver)".

User A (Sender) section:

- Private Key (ASCII decimal): A text input field.
- Public Value: A text input field.
- Message to send: A large text area with a vertical scrollbar.

User B (Receiver) section:

- Private Key (ASCII decimal): A text input field.
- Public Value: A text input field.
- Decrypted Message: A large text area with a vertical scrollbar.

Action Buttons:

- "Compute Keys" button (located below User A's fields).
- "Encrypt & Send" button (located below User A's fields).
- "Reset" button (located to the right of the "Encrypt & Send" button).

Shared Data Process section (at the bottom):

- Shared Key: A text input field.
- AES-128 Key: A text input field.
- Sub-messages: A large text area with a vertical scrollbar.
- Encrypted (hex): A large text area with a vertical scrollbar.

Figure 1. The application on first launch, before any input is entered.

STEP 2 • KEY DERIVATION

After entering the two private keys and pressing Compute Keys, the public values are displayed for both users, the shared key is computed independently from each side (both must equal 109), and the 128-bit AES key is shown as 109F109F109F109F.

The screenshot shows a web application titled "Bayocot — Secure Information Exchange". It features two main sections for "User A (Sender)" and "User B (Receiver)".

User A (Sender) inputs:

- Private Key (ASCII decimal): 57
- Public Value: 17
- Message to send: (empty text area)

User B (Receiver) inputs:

- Private Key (ASCII decimal): 167
- Public Value: 75
- Decrypted Message: (empty text area)

Below the input fields are three buttons: "Compute Keys" (highlighted with a blue border), "Encrypt & Send", and "Reset".

Shared Data Process section:

- Shared Key: A: $75^{57} \bmod 199 = 109$ | B: $17^{167} \bmod 199 = 109$ ✓ match
- AES-128 Key: 109F109F109F109F (hex: 31 30 39 46 31 30 39 46 31 30 39 46 31 30 39 46)
- Sub-messages: (empty text area)
- Encrypted (hex): (empty text area)

Figure 2. Public values, shared key (109), and the derived AES-128 key 109F109F109F109F.

STEP 3 • ENCRYPTION AND TRANSMISSION

Entering the plaintext message and pressing Encrypt & Send produces the four sub-messages listed in Section III, with the fourth padded by twelve '@' characters. Each block is encrypted independently with AES-128 and the four ciphertext blocks are concatenated for transmission.

The screenshot shows a web application interface for encryption. At the top, there are three buttons: "Compute Keys", "Encrypt & Send" (highlighted with a blue border), and "Reset". Below the buttons, the "Shared Data Process" section displays the shared key calculation: "Shared Key: A: $75^{57} \bmod 199 = 109$ | B: $17^{167} \bmod 199 = 109$ ✓ match". Below this, the "AES-128 Key" is shown as "109F109F109F109F (hex: 31 30 39 46 31 30 39 46 31 30 39 46 31 30 39 46)". The "Sub-messages:" section lists four messages with their corresponding AES-128 ciphertext blocks in hexadecimal: [1] "The Mandalorian " -> 54 68 65 20 40 61 6E 64 61 6C 6F 72 69 61 6E 20; [2] "Must Always Reci" -> 4D 75 73 74 20 41 6C 77 61 79 73 20 52 65 63 69; [3] "te, This is The " -> 74 65 2C 20 54 68 69 73 20 69 73 20 54 68 65 20; [4] "Way!@@@@@@@@@@@@" -> 57 61 79 21 40 40 40 40 40 40 40 40 40 40 40 40. The "Encrypted (hex):" section shows the concatenated ciphertext blocks: 7A 03 58 E3 0A 9C DD 20 AB 3B C2 B8 55 EF 73 69 34 2F F6 EE F4 A7 15 89 3E 2B B6 F0 A6 99 9B 91 83 D2 2E 8E 41 AA F9 46 6B DC E5 F8 02 0C AB A9 82 69 8B 00 00 48 FF 8B 71 F0 9E 11 6D 92 E6 C3.

Figure 3. The four sub-messages and the concatenated AES-128 ciphertext in hexadecimal.

STEP 4 • DECRYPTION AT THE RECEIVER

On the receiver side the shared key is re-derived independently from User B's private key and User A's public value, the ciphertext is split back into 16-byte blocks, each block is decrypted, and the trailing '@' padding is removed to recover the original plaintext.

The screenshot shows a web application interface for decryption, titled "Bayocot — Secure Information Exchange". It is divided into two main sections: "User A (Sender)" and "User B (Receiver)". In the "User A (Sender)" section, the "Private Key (ASCII decimal)" is 57, the "Public Value" is 17, and the "Message to send" is "The Mandalorian Must Always Recite, This is The Way!". In the "User B (Receiver)" section, the "Private Key (ASCII decimal)" is 167, the "Public Value" is 75, and the "Decrypted Message" is "The Mandalorian Must Always Recite, This is The Way!".

Figure 4. The receiver pane displays the recovered plaintext, matching the original message exactly.

$\S V$
CONCLUSION

...

The implementation reproduces every value specified in the original document: the public values 17 and 75, the shared key 109 derived independently by both parties, the AES-128 key 109F109F109F109F, the four 16-character sub-messages including the '@'-padded final block, and—after a complete encrypt–decrypt cycle—the original plaintext message. The simulation thus demonstrates, end to end, the use of Diffie–Hellman to establish a symmetric key over an untrusted channel and the subsequent use of that key with AES-128 to protect the confidentiality of a transmitted message.

.....

end of document